

**NATIONAL UNIVERSITY OF COMPUTER &
EMERGING SCIENCES ISLAMABAD CAMPUS**

Data Structure Spring2026

ASSIGNMENT 1

“Greatness isn’t given, it’s built. Challenge yourself daily, learn relentlessly, and strive for more because settling is the enemy of progress”

Due Date: 19 Feb 2026

Time: 11:59 pm

Please follow the following submission instructions. Failure to submit according to the format would result in a deduction of marks. Submissions other than Google classroom (e.g., email etc.) will not be accepted. No late submission will be accepted. Correct and timely submission of the assignment is the responsibility of every student; hence no relaxation will be given to anyone.

Instructions and Submission Guidelines:

Total Marks: 125

1. Make sure that you read and understand each and every instruction. If you have any questions or comments you are encouraged to discuss your problems with your colleagues (and instructors) on google classroom.
2. **If there is a syntax error in the code, zero marks will be awarded in that part of the assignment.**
3. Keep a backup of your work always that will be helpful in preventing any mishap and avoid last hour submissions
4. Displayed output should be well mannered and well presented. Use appropriate comments and indentation in your source code.
5. For each question in your assignment, make a separate .cpp/.h file e.g., for question 1, make q1.cpp, q1.h and so on. **Each file that you submit must contain your name, student-id, and assignment on top of the file in the comments (5 marks will be allocated for this).**
6. Combine all your work in one folder. The folder must contain only .cpp files (no binaries, no exe files etc.).
7. Rename the folder as **ROLL-NUM_PROGRAM_SECTION** (e.g., **i230001_DS/AI_A**) and compress the folder as a zip file. (e.g., **i230001_DS/AI_A.zip**). Strictly follow this naming convention, otherwise marks will be deducted.
8. Submit the .zip file on Google Classroom within the deadline.
9. The student is solely responsible for checking the final file for issues like corrupt file, viruses in the file, or mistakenly exe sent. If we cannot download the file from Google Classroom, it will lead to zero marks in the assignment.

Honor Policy

Plagiarism is a grave academic offense that can severely undermine your academic integrity and reputation. Any instance of a student found to have plagiarized their assignment, whether from a peer or external source, will be subject to strict consequences. This may result in a zero score for the current or all assignments, or in severe cases, even a failure in the course. Furthermore, all instances of plagiarism will be promptly reported to the **Disciplinary Committee** for further action.

Note: Start early so that you can finish it on time. Also Please ensure that only concepts covered in class are used for the assignment.

Case Study: CodeCollab — Collaborative Coding Session Manager

CodeCollab is an online collaborative coding platform where developers participate in live coding sessions. Each session consists of a dynamically changing queue of contributors waiting to edit a shared code file. Contributors may be promoted, demoted, removed, or merged into collaborative pairs depending on activity and performance.

The company has hired you to design a backend module that manages the contributor queue using a **Singly Linked List**. The goal is to ensure fair access, efficient restructuring of the queue, and accurate reporting of contributor activity.

Contributor Node Structure

Each node must contain the following attributes:

- ContributorID (unique integer)
- ContributorName (C-style string)
- SkillLevel (Beginner, Intermediate, Advanced)
- ContributionScore (integer)
- WarningCount (integer)
- IsMentor (boolean)
- Next pointer

Students may add additional attributes if required.

Required Functionalities

1. Add Contributor

Insert contributors based on the following rules:

- Mentors must appear at the beginning of the list.
- Advanced contributors must be placed after the last mentor but before others.
- Remaining contributors must be added at the end.

Insertion must be performed through pointer manipulation only.

2. Promote Active Contributors

If a contributor's ContributionScore exceeds a defined threshold:

- Move the contributor closer to the front of the list.
- Promotion must not move the contributor ahead of any mentor.

No new nodes may be created.

3. Remove Inactive Contributors

If WarningCount exceeds a specified limit:

- Remove the contributor from the list.
- Maintain structural integrity of the remaining nodes.

4. Merge Consecutive Collaborators

If two consecutive contributors share the same SkillLevel:

- Combine their ContributionScores.
- Retain the contributor with the higher original score.
- Remove the weaker contributor's node.

5. Reverse Collaboration Segment

Given two ContributorIDs:

- Reverse the portion of the list between them.
- If either ID is missing or out of order, leave the list unchanged.

6. Generate Session Report

The system must generate:

- Total number of mentors.
- Contributor with highest ContributionScore.
- Longest continuous skill-level streak.
- Average score of first half of contributors.

Constraints

- Singly Linked List only.
- No arrays, STL containers, or advanced libraries.
- Iterative logic only.
- Menu-driven interface required.
- Program must handle empty lists and single-node lists.

Grading Rubric

Component	Marks	Criteria
Node and List Design	5	Correct struct definition and pointer management
Add Contributor Logic	7	Correct ordering and insertion rules
Promotion Mechanism	6	Safe node relocation without new allocations
Removal of Inactive Contributors	5	Proper deletion and pointer adjustment
Merge Consecutive Collaborators	6	Accurate detection and merging logic
Segment Reversal	7	Correct pointer-only reversal
Session Report	5	Accurate traversal and computation
Edge Case Handling	3	Empty list, head operations, invalid IDs
Interface and Validation	1	Functional menu and input handling

Question : 02

Marks : 75

Case Study: AI Model Experiment Tracker — Research Pipeline Manager

A research laboratory maintains multiple AI model experiments organized into a sequential pipeline. Each experiment represents a processing step that depends on a previous experiment's output. During experimentation, researchers frequently reorganize the workflow, adjust priorities, remove failed experiments, and optimize execution sequences.

You are hired to implement an **Experiment Pipeline Manager** using a **Singly Linked List** to simulate the workflow.

Experiment Node Structure

Each node must include:

- ExperimentID (unique integer)
- ExperimentName (C-style string)
- ExecutionTime (integer)
- DependencyID (integer indicating prerequisite experiment)
- PriorityLevel (integer scale)
- FailureCount (integer)
- Next pointer

Students may extend the structure if needed.

Required Functionalities

1. Insert Experiment with Dependency Placement

- If DependencyID exists, insert immediately after that experiment.
- If DependencyID is zero or not found, append at the end.
- Detect and prevent circular dependencies through logical traversal.

2. Priority-Based Reordering

When PriorityLevel changes:

- Reposition the experiment earlier or later accordingly.
- Maintain valid dependency order at all times.
- Preserve relative ordering of experiments with equal priority.

3. Retry-Based Promotion

If FailureCount exceeds a specified limit:

- Move experiment closer to the front.
- Ensure dependencies remain satisfied.

4. Detect Workflow Errors

The system must detect:

- Missing dependencies.
- Circular dependencies.
- Experiments appearing before their required dependency.

5. Parallel Execution Candidate Identification

Identify consecutive experiments that:

- Do not depend on each other.
- Could be executed simultaneously.

Return each group as a logical segment.

6. Failure Cascade Removal

If an experiment fails permanently:

- Remove that experiment.
- Remove all experiments that depend on it directly or indirectly.
- Reconnect remaining nodes correctly.

7. Optimization Pass

Reorganize the pipeline so that:

- Higher priority experiments execute earlier.
- Dependency constraints remain valid.
- Execution order is stable for equal priority levels.

Constraints

- Singly Linked List only.
- No arrays, vectors, STL containers, or recursion.
- Pointer manipulation required for all structural changes.
- Menu-driven interface mandatory.
- Input validation required.

Grading Rubric

Component	Marks	Criteria
Data Structure Design	6	Correct node design and pointer use
Dependency-Based Insertion	10	Correct placement and validation
Circular Dependency Detection	7	Logical detection without auxiliary structures
Priority Reordering	10	Stable pointer-based reordering
Retry Promotion	7	Safe relocation maintaining dependencies
Workflow Error Detection	8	Correct identification of structural issues
Parallel Candidate Identification	7	Logical analysis of independence
Failure Cascade Removal	10	Accurate multi-node deletion
Optimization Pass	8	Stable reordering under constraints
Edge Case Handling	2	Empty lists, head operations, invalid inputs

Question : 03 (Bonus)

Walk into your demo with a smiling face and earn 5 bonus marks.

*****Good Luck*****